The core principle in large-scale system design is:

> **Do not design for "the cloud provider will be available." Design for failure of every infrastructure layer, and make the system continue operating at the highest possible level of service.**

For interview purposes, this is usually called **fault tolerance + high availability + disaster recovery + resilience engineering**.

A useful way to reason about it is to build a **failure-domain hierarchy**:

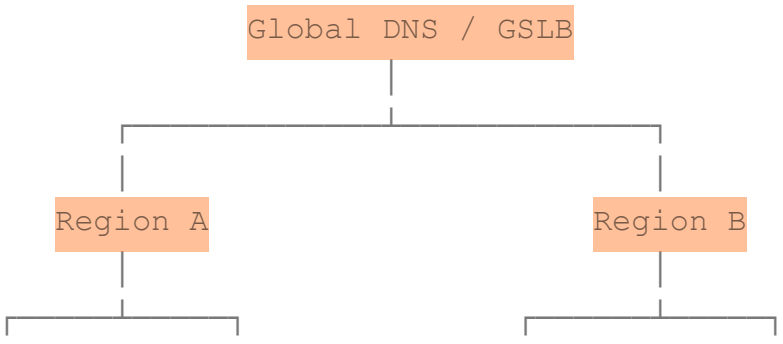
```
Application
 ├── Process failure
 ├── Pod/container failure
 ├── K8s node failure
 ├── Rack / host-group failure
 ├── Availability Zone failure
 ├── Region / Data Center failure
 └── Cloud-provider / control-plane failure
```

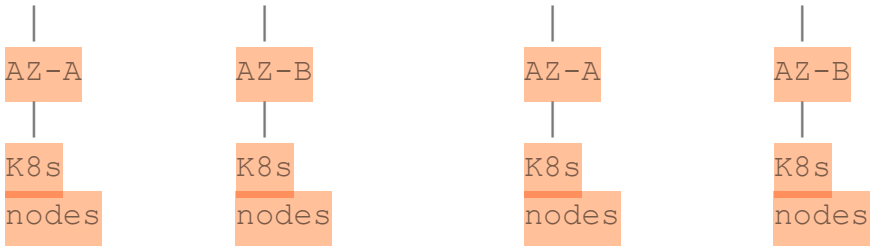
Your architecture should decide **what happens at each level**.

---

1. Start with failure domains

Suppose you deploy:





You don't necessarily need multi-region for every system.

The important question is:

> **What is the largest failure I need to survive?**

For example:

Failure	Typical response
Process crashes	Restart
Pod dies	Kubernetes reschedules
Node dies	Pod moves to another node
Several nodes fail	Capacity distributed across node pools/AZs
AZ fails	Traffic moves to other AZs
Region fails	Traffic moves to another region
Database primary fails	Automatic/controlled failover
Entire provider fails	Multi-cloud / alternate provider
Corrupted data	Backup + point-in-time recovery
Ransomware/deletion	Immutable/offline backup

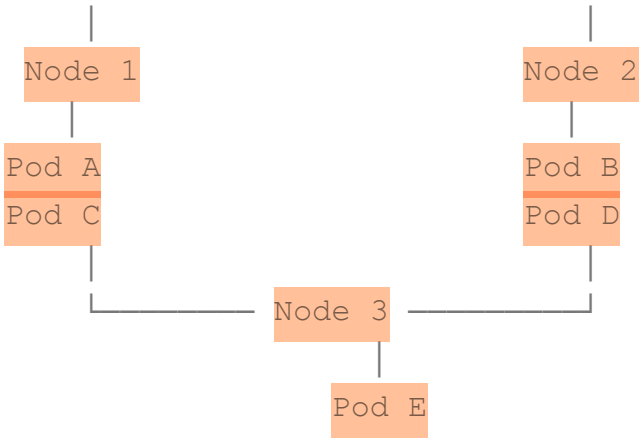
This is the foundation of the design.

---

2. Kubernetes node failure

Consider:





If Node 1 dies:

```
Node 1 ❌

Pod A ──────▶ Node 2
Pod C ──────▶ Node 3
```

Kubernetes can recover the workloads **if you designed the workloads correctly**.

Use multiple replicas

```
replicas: 5
```

rather than:

```
replicas: 1
```

But replicas alone aren't enough.

---

3. Don't put all replicas on the same node

This is a common interview trap.

Bad:

```
Node 1
 ├── Pod A
```

```
├── Pod B
└── Pod C
```

Node failure:

```
Node 1 ❌

A ❌
B ❌
C ❌
```

Instead:

```
Node 1          Node 2          Node 3
 ├─ Pod A        ├─ Pod B        ├─ Pod C
```

Use:

- Pod anti-affinity
- topology spread constraints
- node affinity
- PodDisruptionBudget
- multiple node pools

Conceptually:

```
Replica distribution:

                AZ-A          AZ-B          AZ-C

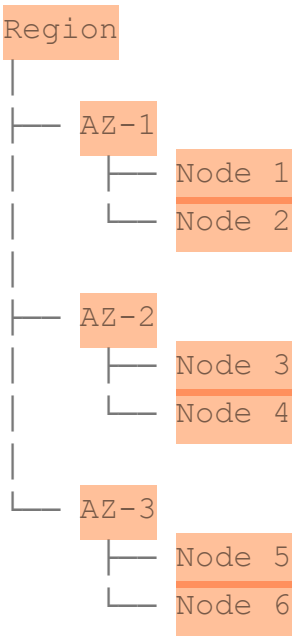
Node            N1  N2        N3  N4        N5  N6
                |             |             |
Pods            A             B             C
                D             E             F
```

Now losing one node doesn't eliminate all replicas.

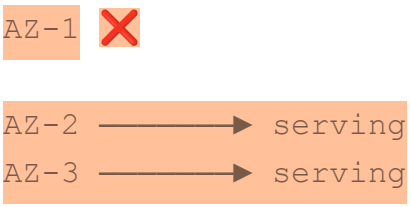
---

4. Availability Zones are the next boundary

Suppose:



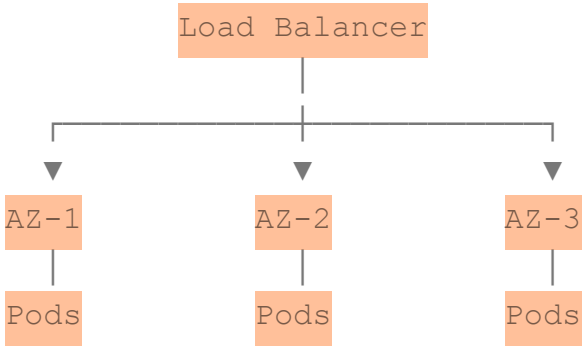
If AZ-1 disappears:



This requires more than Kubernetes.

Your **load balancer must also understand the failure domain**.

For example:



Health checks prevent traffic from going toward failed infrastructure.

---

5. Capacity planning is critical

A system isn't highly available merely because it has replicas.

Imagine:

```
AZ-1: 40% capacity
AZ-2: 40%
AZ-3: 20%
```

Normal:

```
100% capacity available
```

AZ-1 fails:

```
60% capacity remains
```

But if you need 80% capacity to handle production traffic:

```
60% < 80%

SYSTEM OVERLOADED
```

Therefore:

> **Design capacity for the loss of your largest expected failure domain.**

For example:

```
Normal utilization = 50%

AZ failure
    ↓
Remaining infrastructure = 66%

Still enough capacity
```

This is why large systems frequently maintain **headroom**.

---

6. Stateless services should be easy to replace

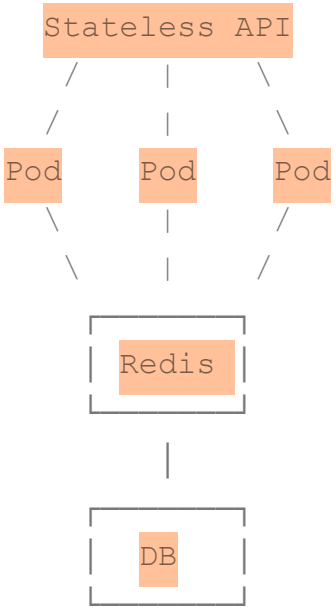
A very important architecture principle:

> **Compute should be disposable; state should be durable.**

Instead of:

```
Application
   │
   ├── local files
   ├── local session
   ├── local cache
   └── local database
```

prefer:



If a pod disappears:

```
Pod ❌

No important state is lost.
```

Kubernetes simply creates another pod.

---

7. Separate compute failure from state failure

This is where system design becomes significantly harder.

Compute:

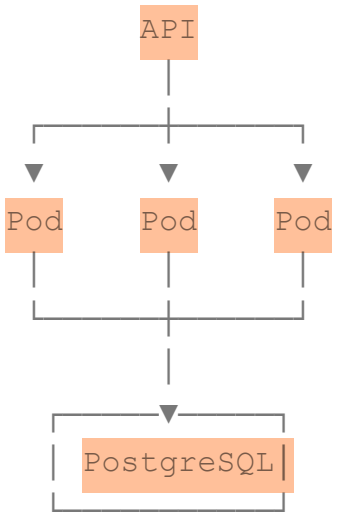
`Kubernetes`

is generally easier to make highly available than:

`Database`

because databases contain state.

For example:



If PostgreSQL dies, adding another API pod doesn't help.

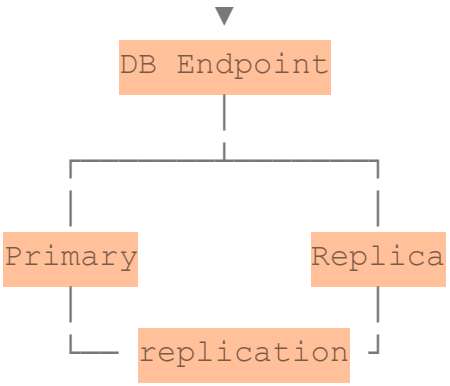
Therefore stateful systems need their own HA architecture.

---

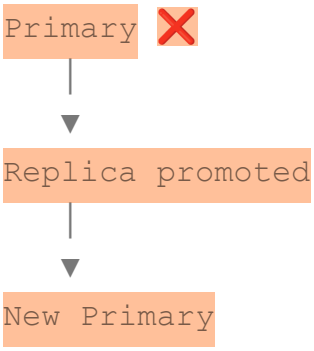
8. Database HA

A common architecture:





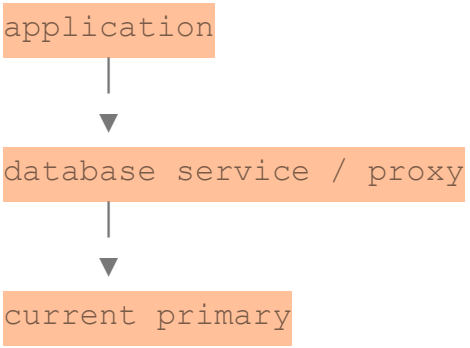
Primary fails:



The application should connect through a stable endpoint rather than hard-coding:

```
postgres-primary-1
```

For example:



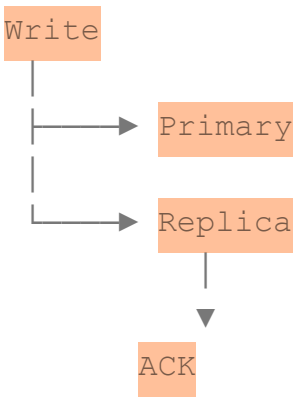
This allows the database topology to change without requiring application changes.

---

9. Synchronous vs asynchronous replication

This is an important interview topic.

Synchronous



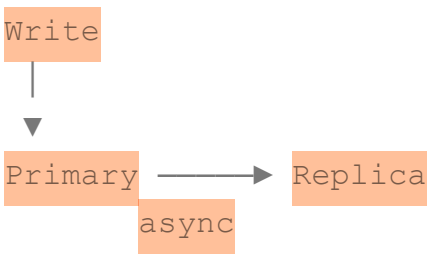
Advantages:

- lower data-loss risk
- stronger consistency

Disadvantages:

- higher latency
- replication failure can affect writes
- difficult across distant regions

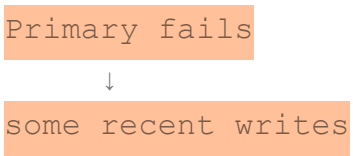
Asynchronous



Advantages:

- lower latency
- works better across regions

Disadvantage:



```
      ↓
may not have reached replica
```

This is the **RPO trade-off**.

---

10. RPO and RTO

These two concepts are fundamental to disaster recovery.

RPO — Recovery Point Objective

> How much data can we afford to lose?

Example:

```
RPO = 5 minutes
```

means:

> In the worst case, we can tolerate losing up to approximately 5 minutes of acknowledged data, depending on the recovery design.

RTO — Recovery Time Objective

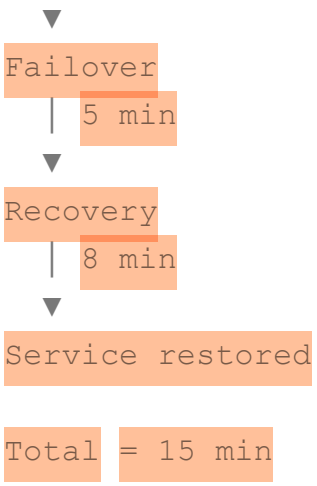
> How long can the system be unavailable?

Example:

```
RTO = 15 minutes
```

means:

```
Failure
   |
   ▼
Detection
   | 2 min
```

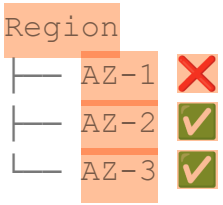


---

11. AZ failure vs region failure

These are fundamentally different.

AZ failure



Multi-AZ architecture can usually handle this.

Region failure

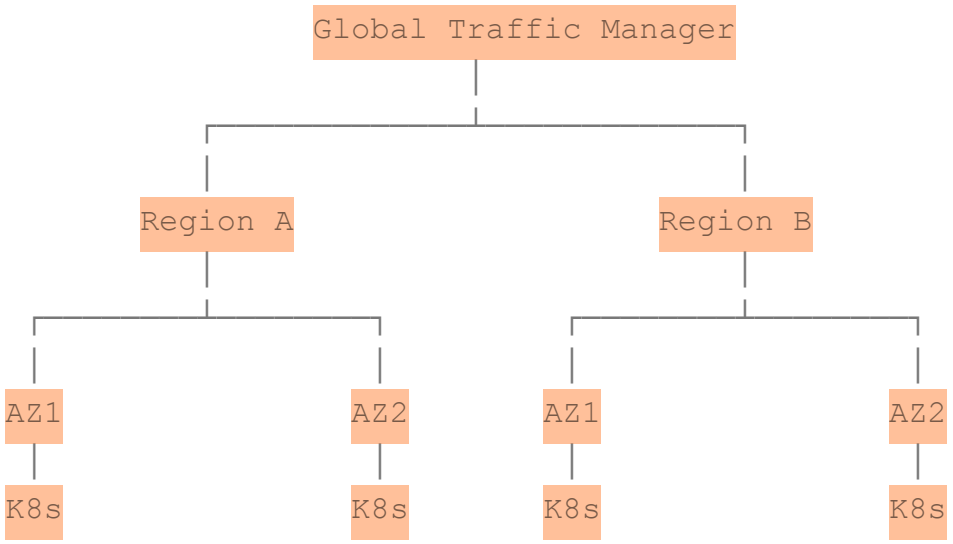


Now you need **multi-region architecture**.

---

12. Multi-region architecture

A typical architecture:



Normal:

```
Users ———▶ Region A
```

Region A fails:

```
Users ———▶ Region B
```

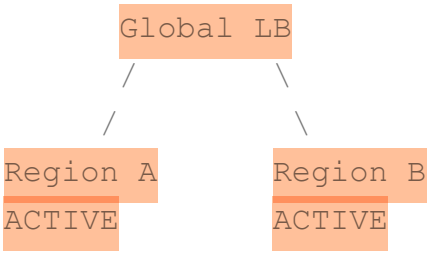
But this requires:

- global traffic management
- health checks
- replicated state
- replicated configuration
- replicated secrets
- replicated artifacts
- database failover
- operational procedures

---

13. Active-active vs active-passive

Active-active



Both serve traffic.

Advantages:

- better utilization
- fast failover
- potentially lower RTO

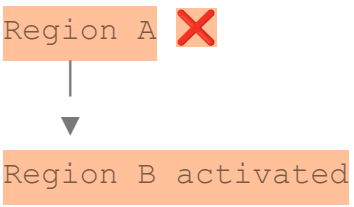
Challenges:

- data consistency
- conflict resolution
- distributed transactions
- operational complexity

Active-passive



If A fails:



Advantages:

- simpler
- easier consistency model

Disadvantages:

- idle/underutilized capacity
- potentially slower failover
- failover process must actually be tested

---

14. DNS isn't enough

A common interview answer is:

> "Use DNS failover."

That's incomplete.

DNS caching means clients may continue using the failed endpoint.

Instead, large systems often combine:

```
Global traffic manager
        |
        ├── health checks
        ├── routing
        ├── regional failover
        └── load balancing
```

Possible mechanisms include:

- DNS-based routing
- Anycast
- global load balancers
- CDN edge routing
- application-level routing

The exact mechanism depends on the cloud/provider and latency requirements.

---

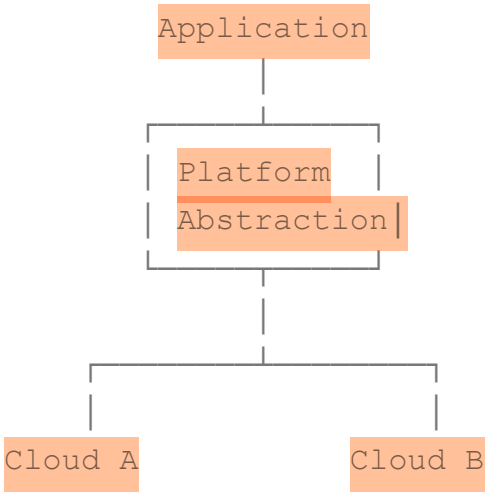
15. Don't depend on one cloud service

Suppose your application uses:

```
Cloud Provider
├── Kubernetes
├── Database
├── Object Storage
├── Queue
├── Cache
├── DNS
└── Identity
```

If the provider has a major outage, all of these could potentially become unavailable.

A more resilient architecture can abstract critical dependencies:



But **multi-cloud is not automatically better**.

It introduces:

- duplicated infrastructure
- operational complexity
- networking complexity
- data replication complexity
- different service semantics
- higher cost
- more difficult observability
- more difficult security management

So the decision should be driven by the required failure boundary and RTO/RPO.

---

16. Protect against control-plane failures

There's another subtle failure mode.

Suppose:

```
Kubernetes control plane ❌
```

Your existing workloads might continue running.

But you may be unable to:

- deploy
- scale
- reschedule
- modify services
- perform normal cluster operations

Therefore critical systems should consider:

```
Application data plane
          │
          ├── independent capacity
          │
          └── independent operational path
```

This distinction is important:

> **Control plane failure does not necessarily equal data plane failure.**

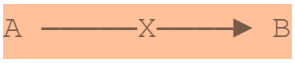
---

17. Queue-based decoupling

Suppose Service A calls Service B synchronously:

```
A ─────HTTP─────▶ B
```

If B is down:

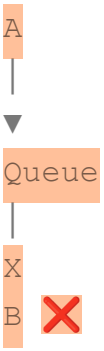


Now A may also fail.

Instead:

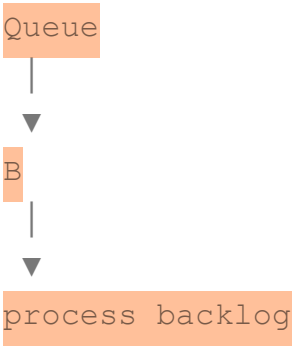


If B goes down:



Messages remain queued.

When B recovers:



This provides **temporal decoupling**.

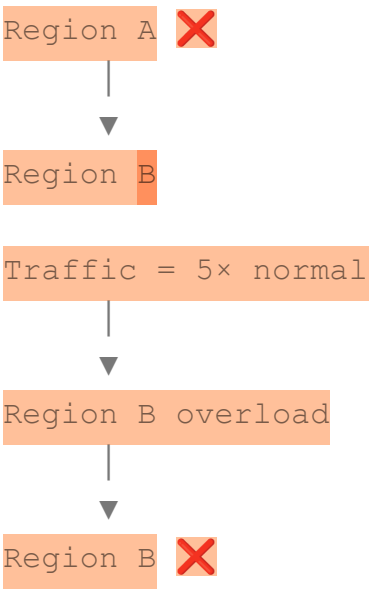
Kafka, RabbitMQ, cloud queues, etc. can play this role depending on requirements.

---

18. Backpressure is essential

Suppose Region B suddenly receives traffic from Region A.

Without protection:

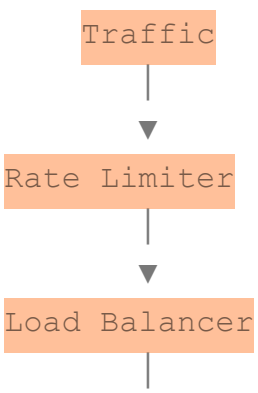


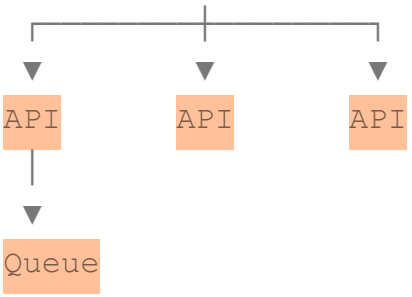
This creates a cascading failure.

Therefore implement:

- rate limiting
- admission control
- circuit breakers
- bounded queues
- load shedding
- bulkheads
- autoscaling
- priority queues

For example:





---

19. Graceful degradation

A highly resilient system doesn't necessarily provide **100% functionality** during every failure.

Instead:

> **Preserve the most important functionality and degrade optional functionality.**

Example:

```
E-commerce

Critical:
✓ Product browsing
✓ Checkout
✓ Payment

Non-critical:
✗ Recommendations
✗ Personalized ranking
✗ Analytics
✗ Promotional suggestions
```

If recommendation infrastructure fails:

```
Recommendations ❌

Checkout continues.
```

This is much better than:

```
Recommendation service ❌
          ↓
```

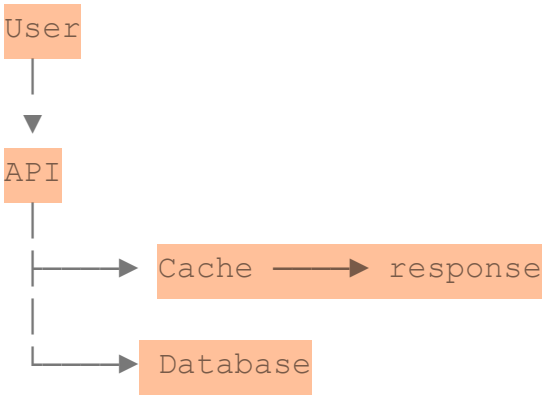
```
Entire website ❌
```

---

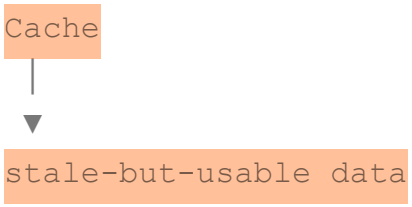
20. Caching can protect the system

Suppose the database becomes unavailable.

You might still serve cached/read-only information:



If DB fails:



This is a conscious consistency trade-off.

---

21. Configuration must also be redundant

People often replicate application servers but forget:

```
configuration
secrets
certificates
feature flags
container images
deployment manifests
```

```
Terraform
CI/CD
```

Imagine:

```
Region A ❌
```

and Region B exists, but:

```
Container image → only Region A registry
Secrets         → only Region A
Terraform state → only Region A
Configuration   → only Region A
```

Your "DR region" isn't actually recoverable.

Therefore:

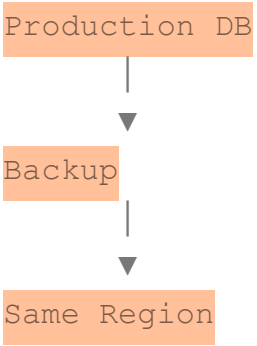
> **Everything required to reconstruct the service must itself survive the failure domain.**

This leads to **Infrastructure as Code** and automated environment reconstruction.

---

22. Backups must be outside the failure domain

Bad:



If the region is destroyed:

```
DB ❌
Backup ❌
```

Better:

```
Production DB
      |
      ▼
Backup
      |
      ▼
Different region
```

For serious disaster recovery:

```
Primary
  |
  ├── replication
  |
  ├── snapshots
  |
  └── immutable backups
                |
                ▼
        Separate failure domain
```

The strongest designs also protect against accidental deletion and corruption with immutable/versioned backups.

---

23. Observability must be multi-region

Don't put your monitoring system only inside the region it monitors.

Bad:

```
Region A
  ├── Application
  └── Monitoring
```

```
Region A ❌
    ↓
Monitoring ❌
    ↓
You don't know what happened
```

Better:



Monitor:

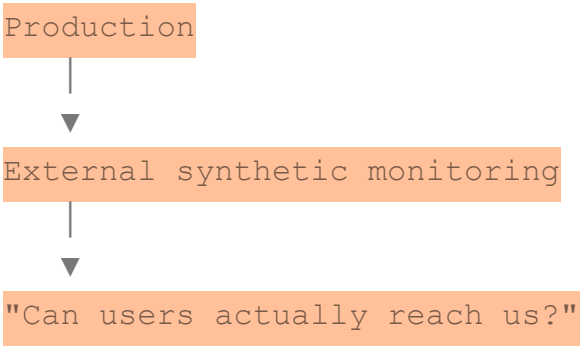
- availability
- latency
- error rates
- saturation
- queue depth
- replication lag
- failover state
- capacity
- dependency health

---

24. Failure detection must be independent

Don't depend entirely on the same infrastructure you're testing.

For example:



This detects failures that internal monitoring might miss.

---

25. Chaos engineering

You shouldn't merely assume:

> "Kubernetes will recover."

Test it.

For example:

```
Kill pod
    ↓
Does service recover?

Kill node
    ↓
Does service recover?

Remove AZ
    ↓
Does service recover?

Break database primary
    ↓
Does failover work?

Block network
    ↓
Does circuit breaker work?

Kill entire region
    ↓
Does DR activate?
```

This is **failure injection / chaos engineering**.

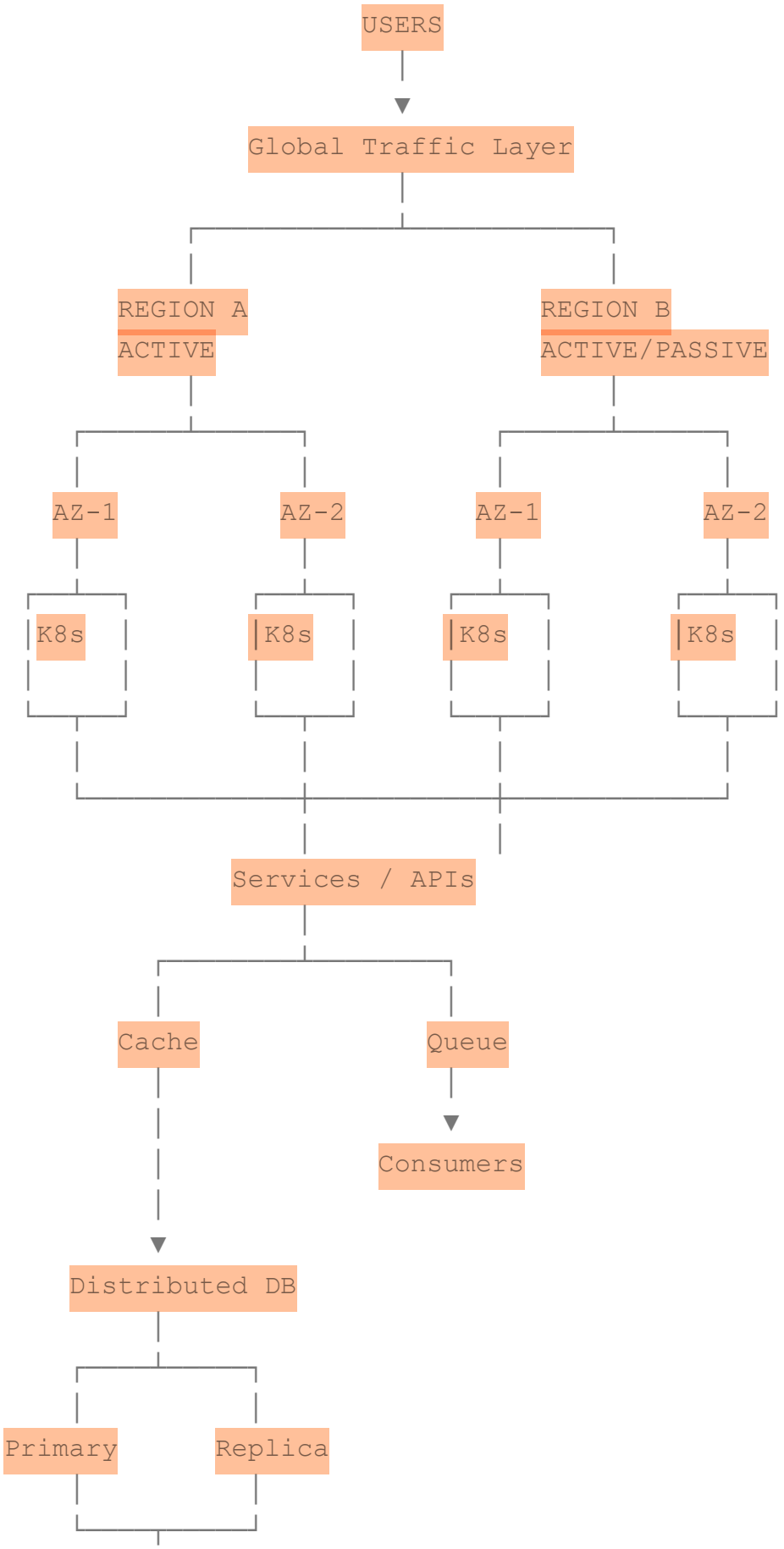
The most important rule:

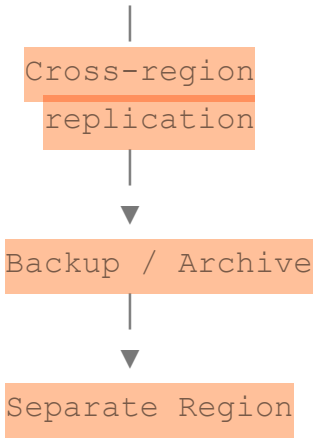
> **A disaster recovery plan that has never been tested is an assumption, not a capability.**

---

26. The complete architecture

For a serious large-scale platform, you might arrive at something like:





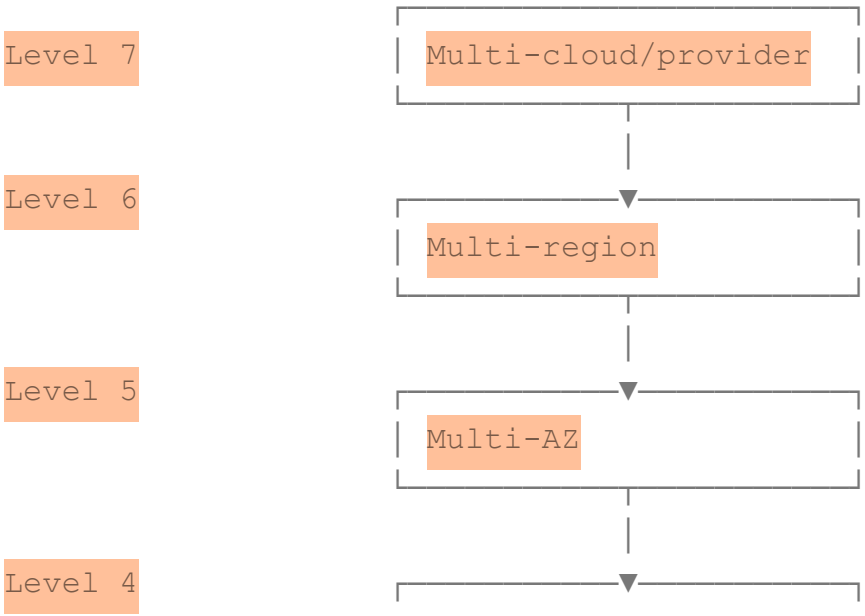
Alongside this:

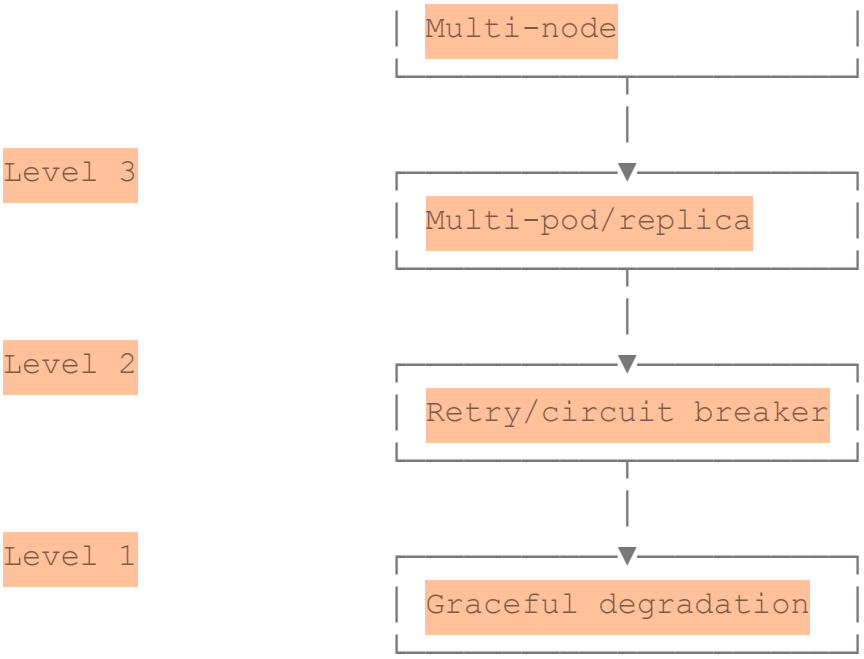
- Observability
- Security
- IAM
- Secrets
- Configuration
- IaC
- CI/CD
- Feature flags
- Disaster recovery
- Chaos testing

---

27. A useful resilience hierarchy

For interviews, I recommend memorizing this hierarchy:





But there is another dimension:

```
Availability
      +
Durability
      +
Recoverability
      +
Observability
      +
Operability
```

You need all five.

---

28. Don't automatically choose multi-cloud

This is an important senior/staff-level distinction.

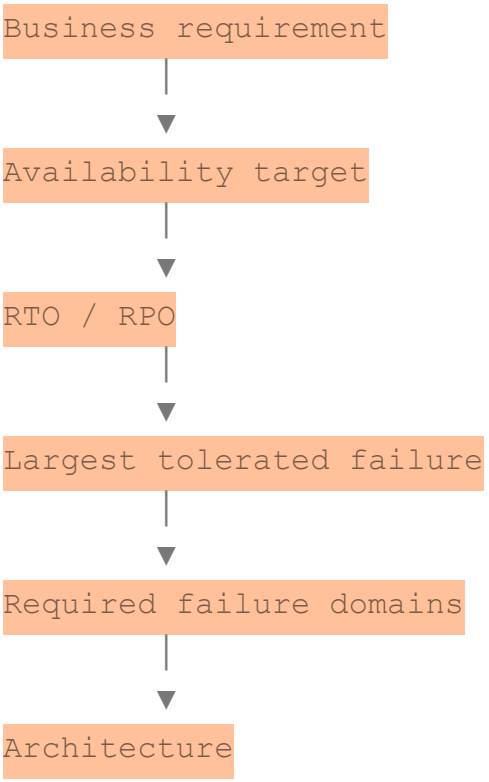
Suppose the requirement is:

> 99.99% availability.

You shouldn't immediately say:

> "Use two clouds."

Instead work backward:



For example:

Requirement

```
99.9%
RTO = 1 hour
RPO = 15 min
```

Maybe:

```
Multi-AZ + backups + DR region
```

is sufficient.

Whereas:

```
99.999%
RTO = <5 min
RPO ≈ 0
provider-level failure must be tolerated
```

could justify substantially more elaborate architecture.

---

29. The CAP theorem appears here

When you introduce multi-region state, you're entering distributed-systems territory.

You have to reason about:

```
Consistency
Availability
Partition tolerance
```

During a network partition:

```
Region A  ←──X──→  Region B
```

what should happen?

Option A:

```
Preserve consistency
        ↓
reject some writes
        ↓
reduced availability
```

Option B:

```
Preserve availability
        ↓
accept writes independently
        ↓
possible conflicts
```

There isn't a universally correct answer.

The business requirement determines the trade-off.

---

30. A very strong interview framework

When asked:

> **"How would you design the system to survive cloud infrastructure failures?"**

Walk through these questions in order:

1. What failure are we surviving?

```
Pod?
Node?
AZ?
Region?
Provider?
Data corruption?
```

2. What availability is required?

```
99.9?
99.99?
99.999?
```

3. What are RTO/RPO?

```
RTO = ?
RPO = ?
```

4. Where is state?

```
DB
Cache
Object storage
Queues
Sessions
Files
```

5. How is state replicated?

```
sync?
async?
multi-region?
```

6. How does traffic fail over?

```
LB
GSLB
DNS
Anycast
```

7. How does compute recover?

```
K8s
autoscaling
replicas
anti-affinity
multi-AZ
```

8. How do we prevent cascading failure?

```
timeouts
retries
backoff
circuit breakers
bulkheads
rate limiting
load shedding
queues
```

9. How do we degrade?

```
Which features can be sacrificed?
```

10. How do we recover?

```
backup
restore
DR
IaC
runbooks
automation
```

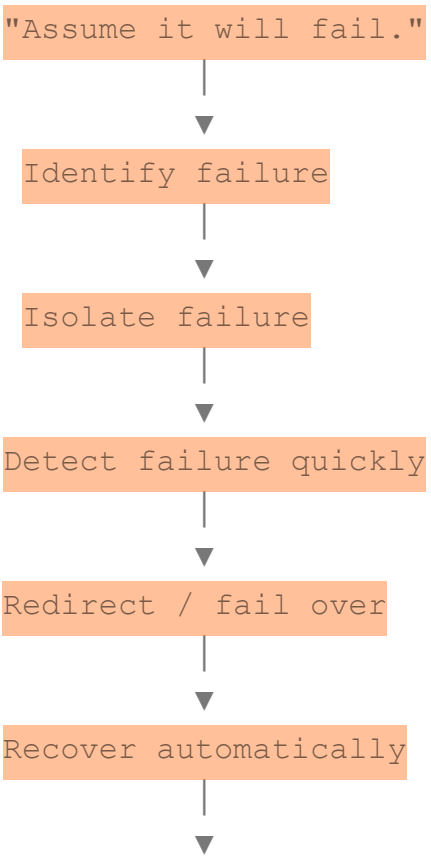
11. How do we know it works?

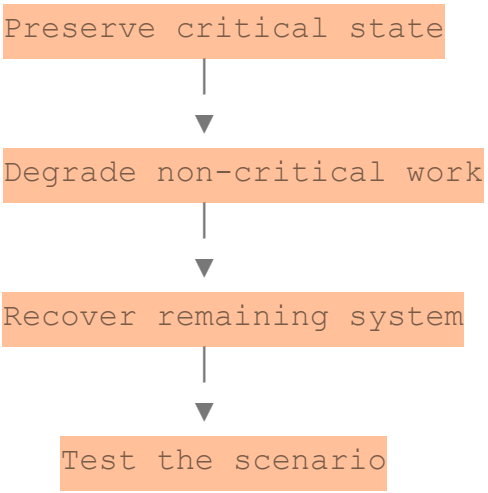
```
monitoring
synthetic checks
chaos testing
DR drills
```

That sequence demonstrates **Staff/Principal-level system-design thinking**, because you're designing around failure domains rather than simply listing technologies.

The key mental model

Ultimately, think of cloud infrastructure like this:





And the most important architectural principle is:

> **Replicate across independent failure domains, keep compute stateless where possible, make state durable and recoverable, decouple components, design explicit failover/degradation behavior, and continuously test the failure scenarios you claim to survive.**

For your **MAANG+ system-design preparation**, this topic connects directly to **Kubernetes HA, multi-AZ/multi-region architecture, distributed databases, Kafka/RabbitMQ, caching, circuit breakers, graceful degradation, CAP theorem, RTO/RPO, disaster recovery, chaos engineering, and multi-cloud design**.